Do the Numbers Hold on Real Hardware? A Reproducibility and Measurement Audit of a Low-Cost Hybrid ECC Authentication Protocol for IoT

Jay Kerkar, Chaitanya Kelkar, Chirag Poornamath, Yash Kolekar
Department of Information Technology
Vivekanand Education Society's Institute of Technology
Mumbai, India

Dr. Shanta Sondur
Project Guide
Department of Information Technology
Vivekanand Education Society's Institute of Technology
Mumbai, India

Abstract—Al-Rasheed et al. proposed a two-message hybrid ECC/symmetric-key mutual-authentication protocol for the Internet of Things and reported roughly 17%, 21% and 12% lower communication, processing and storage cost than prior schemes. Those figures rest on OMNeT++ simulation and assumed per-operation times, and no code or scenario was published. We audit what can be checked. (i) Their stated percentages cannot be derived from their own tables or figures under any definition we tried, and their tables contradict their text (four messages and 1024 bits versus two messages and 264 bits). (ii) We could not reproduce their processing figure (Fig. 1) from published information, and we make no claim about whether it is correct. (iii) We implement the device side in C, interoperate it with an independent Python server on an ESP8266, and time the underlying primitives on the ESP8266, an AWS Graviton2 core and a laptop. P-256 scalar multiplication takes 276 ms on the ESP8266, sixteen times the assumed 17.1 ms, while symmetric operations are five to twelve times faster than assumed. (iv) Recomputing their cost model with measured values, the protocol is about 60–2 500× cheaper than the ECC-based schemes it is compared with. Against the hash-only schemes in their Table V, it costs more once the session-key derivation required by its own equations is counted (about 4–14× on the ESP8266); this holds for the two hash-only schemes whose operation counts we verified against independent sources, and the comparison otherwise depends on that table's counts. We also show that the scheme lacks forward secrecy, uses no elliptic-curve operation online, and has a server-side pseudonym lookup that grows with the number of devices. All code and raw data are open.

Index Terms—Internet of Things, mutual authentication, elliptic curve cryptography, reproducibility, microcontroller benchmarking, lightweight security.

I. Introduction

Lightweight authentication schemes for the Internet of Things (IoT) are usually compared by counting cryptographic operations and multiplying by assumed per-operation times. The conclusion of such a comparison then depends on a few constants that are rarely measured on the class of device the scheme targets. Al-Rasheed et al. [1] propose a hybrid scheme in which elliptic-curve cryptography is confined to an offline registration phase and every online authentication uses a two-message nonce-based symmetric handshake. They report about 17%, 21% and 12% lower communication, processing and storage cost than ten prior schemes, using OMNeT++ simulation and the reference times of their Table III (an ECC scalar multiplication at 17.1 ms, a symmetric encryption or decryption at 5.6 ms, a SHA-1 hash at 0.32 ms).

We test what can be tested. We do not claim that the authors' simulated results are wrong: where their evidence is unavailable we say so and stop. Our contributions are:

1) A consistency and reproducibility audit of the paper's tables, equations and figures, including checks of the cost rows against three of the original papers it draws on (Sections IV-A and IV-B).
2) Measured primitive costs on a 32-bit microcontroller (ESP8266), an AWS Graviton2 core and a laptop, using one C source on every platform with checksum-verified outputs, and a recomputation of the paper's cost model with a sensitivity analysis (Sections IV-C and IV-D).
3) Protocol-level findings that bear on how the scheme should be compared: no forward secrecy, no online elliptic-curve operation, and a server lookup cost that grows with the number of devices (Section IV-E).
4) An open implementation and benchmark: a Python reference of the protocol, a portable C device core,

ESP8266 firmware, and the audit scripts.

II. The Base Protocol and Its Evidence

In the offline phase a trusted authority (TA) derives a long-term secret $\lambda_X = H_1(\mathrm{ID}_X \,\|\, s) \bmod n$ for each device and server (Eq. 8 of [1]) and distributes peer secrets, so that both sides can compute a pairwise root $k^\star_{ij} = \mathrm{KDF}(\lambda_{V_i} \,\|\, \lambda_{R_j})$ (Eq. 9). Online, a device sends $M_1 = \langle \mathrm{PID}_{V_i,t}, C_1, \mu_1 \rangle$ with $\mathrm{PID} = H_2(\lambda_{V_i} \,\|\, t)$, $C_1 = \mathrm{Enc}_{k^\star}(\mathrm{PID} \,\|\, N_i \,\|\, t)$ and a MAC μ_1; the server replies with M_2, and both derive the session key $K = \mathrm{KDF}(k^\star \,\|\, N_i \,\|\, N_j \,\|\, t)$ (Eq. 16). The instantiation notes name AES-GCM [13], HKDF [12] and SHA-256. The online cost is stated as one KDF, one encryption and one MAC on the device (Eq. 22).

Cost claims rest on Tables IV–VI, which compare the proposed scheme with schemes labelled [2]–[8] and others, and on Figs. 1–3, which compare against four different schemes. Processing cost (Table V) is a per-role count of hash (T_H), symmetric ($T_{SE/D}$), ECC scalar-multiplication (T_{ECM}) and fuzzy-extraction (T_{fe}) operations, multiplied by the Table III constants.

III. Methodology

A. Audit

Tables III–VI were transcribed from 300-DPI renders of the published PDF and re-read at 600 DPI. Tables III, IV and VI were additionally checked by optical character recognition (every cell matched); Table V, whose mathematical typography defeats OCR, was checked by two manual cell-by-cell readings. Bar heights in Figs. 1–3 were read by eye from 600-DPI renders (reading error about ± 0.2 axis units). For each figure and table we computed the reduction of the proposed scheme under nine definitions (against the best, mean, median and worst competitor, averaged over the x-axis, over all scheme/x pairs, at the first and last point, and by ratio of sums).

B. Implementation

We implemented the nonce-based model of the paper's Section III in Python. (Section II of the paper describes a different exchange, Algorithms 1–2, whose hash digest and random number are static and carry no per-session nonce, so a recorded exchange can be replayed. Section III has the nonces, the security sketch and the cost equation Eq. 22, so it is the version we implement; our trusted authority derives λ_X locally as in Eq. 8 and does not run the ECDH registration of Eqs. 1–5.) The implementation uses the cryptography package (AES-256-GCM, HKDF-SHA256, SHA-256), with a length-prefixed JSON/base64 wire format. We then wrote a portable C device core on BearSSL [10] with the identical wire format and ran it (i) natively against the unmodified Python server (12/12 handshakes) and (ii) on a NodeMCU ESP8266 (Tensilica L106, 80 MHz, about 80 KB RAM, no cryptographic hardware) over Wi-Fi (20/20 handshakes). In every case the session-key fingerprints on the two sides were identical and a replayed M_1 was rejected. On the ESP8266 a handshake costs 1.67 ms to build M_1 and 3.59 ms to verify M_2 and derive the key (5.26 ms of device compute, mean of 20), and 52 ms end to end over Wi-Fi; M_1 is 216 bytes.

C. Primitive benchmark

A single C benchmark, built from the same source on every platform, times SHA-1 and SHA-256, HMAC, HKDF-SHA256, AES-256-GCM (with the key schedule paid or cached; three BearSSL AES implementations: bitsliced constant-time aes_ct, small-table aes_small and fast-table aes_big), bare AES and AES-CTR, and P-256 and X25519 scalar multiplication. Platforms: (a) ESP8266 at 80 and 160 MHz, Wi-Fi off, cycle-counter timing; (b) an AWS t4g.nano (Graviton2/Neoverse-N1), one pinned core; (c) an Apple-silicon laptop; (d) on the laptop, OpenSSL [11] libcrypto, which uses the hardware AES and SHA instructions, timed per message (fresh IV, tag, cached key) with low-level SHA-1 and a low-level HKDF verified equal to OpenSSL's. Each value is the median of 100 runs (24 for elliptic-curve operations). Every output (hashes, ciphertext and tags, curve points) is checksummed; all 41 digests are identical on all BearSSL platforms and no operation reported failure.

D. Recomputing the cost model

We keep the paper's per-role operation counts (Table V) and replace the Table III times by measured ones. Because Table V's count for the proposed scheme ($2T_H + 2T_{SE/D}$ per side) omits the session-key derivation of Eqs. 16 and 22, we evaluate four readings: V1 Table V as printed; V2 Table V plus one KDF per side; V3 Eq. 22 with an AEAD (device: one KDF and one AEAD operation; server: one KDF and two AEAD operations); V4 Table V's "$2T_H$" read as the pseudonym hash plus the KDF (the reading most favourable to the paper). We also sweep how $T_{SE/D}$ is defined (full GCM, GCM with cached key, CTR without MAC, one AES block) and T_H (one or two SHA-1 blocks). Competitor protocols are not re-implemented: their cost is the paper's operation counts times our measured times.

IV. Results

A. Consistency of the paper's evidence

Items 1–5 follow from the paper's own text, tables and figures alone; items 6–7 compare its rows with the original papers.

1) The headline percentages are not derivable. Under none of nine definitions do Tables IV–VI or Figs. 1–3 give 17%, 21% or 12% (within one point). Table IV gives 36.0% fewer bits than the best competitor and 57.5% fewer than the mean; Table VI gives 10.1% (device) and 33.3% (server) less storage than the best; the three figures give 24–49%, 19–31% and 7–25% depending on definition.

2) Tables contradict the text. The text, Eq. 21 and Section V-B state two messages and 264 bits; Table IV lists the proposed scheme with four messages and 1024 bits. Two 128-bit messages total 256, not 264, and a SHA-256 pseudonym (256 bits) plus a 128-bit GCM tag already exceed 264 bits in M_1 alone.
3) Under the paper's own constants it is not the cheapest. With Table III, Table V costs the proposed scheme at 23.7 ms; three rows costed with hash operations only come to 6.1, 7.0 and 7.0 ms (all roles).
4) Internal inconsistencies. Table V charges $2T_H + 2T_{SE/D}$ per side whereas Eq. 22 states one KDF, one encryption and one MAC; Algorithms 1–2 describe a different (MAC-address and hash-digest) protocol than Sections III–V; Table VI is captioned "communication cost" but reports storage; two schemes share reference number [17]; Table II lists servers as "5% (one per hundred devices)" (5% is one per twenty) and a "packet size" of 128 Kbps (a rate).
5) The figures disagree with the text. Fig. 1 is captioned "processing" but its axis reads "communication overhead"; Figs. 1–2 plot against "vehicles in VANET" (300–1800) although the paper targets static IoT devices; Fig. 2 plots 22–41 "bits" against 264 or 1024 in the text and table, rising with the number of vehicles for a fixed two-message protocol; two series in Fig. 1 are clipped at the axis limit; and Figs. 1–3 compare against four schemes (Gope, Gupta, Hassan, Zhanfei) that appear in none of Tables IV–VI and in neither the reference list nor the text.
6) Row labels do not match the reference list. The rows "Cheng and Le [14]", "Jiang et al. [15]" and "Shuai et al. [16]" carry numbers that, in the paper's own reference list, belong to Wang et al. (emergency-vehicle VANET authentication), Wen et al. (an edge-AI survey article) and Jiang et al. (a chaotic-map three-factor scheme); "Wazid et al. [17]" points to Challa et al. Against the original papers we had available (Chang and Le [9], Dhillon and Kalra [4], Jiang et al. [5]): the "Cheng–Le" row matches Chang and Le's second protocol with the user and gateway hash counts exchanged; the rows "Jiang [15]" (7/5/10 hash operations) and "Shuai [16]" (hash plus one ECC operation per role) match none of the available papers, and Jiang et al.'s chaotic-map scheme contains no ECC operation. We could not identify the source of either row.
7) The cost environment differs from that of the compared papers. The paper's Table III sets the chaotic-map cost T_C equal to T_H (0.32 ms); Jiang et al. state $T_H = 0.20$ ms and $T_C = 32.40$ ms (a factor 162). Their scheme is therefore not hash-only, and costing it with its own constants changes it from cheaper than the proposed scheme to far dearer (Section IV-D). Separately, the paper's Table V lists Turkanović et al. as 7/7/5 hash operations (user/device/server), whereas the two papers that tabulate it [4], [9] give 7/5/7; the total is unchanged but device-only and server-only comparisons are not.

B. Fig. 1 could not be reproduced

Fig. 1 is the paper's only processing-cost evidence that is not a restatement of Table V, and it underlies the 21% claim. We tried to reproduce it, with a fixed budget, and could not. The paper states that costs "were computed in OMNeT++" (Section V) and later that the scheme was simulated in SUMO (Section VI), but gives no scenario, mobility trace, message sizes, timing model or code, and the article has no data-availability statement. We searched the web for a repository, dataset or supplementary material (two passes, October 2026; the publisher's page was not accessible to us) and found none, and we have not contacted the authors. The simulation is therefore not publicly available; we do not claim it does not exist.

We also tried the one thing available to us: a deterministic cost model of the kind the paper describes (operation counts times constants), which gives a per-authentication cost independent of the number of vehicles, so its curves would be flat. The curves digitised from Fig. 1 rise by 5–21 axis units over 300–1800 vehicles, and a flat or two-parameter affine fit leaves residuals of 10–36 and up to 20 times our reading error respectively (research/paper_audit/fig1_repro.py). We read this only as a statement that Fig. 1 depends on simulation effects, such as queueing or mobility, that are not documented; it is not evidence that the figure is wrong. The other series belong to uncited schemes, the y-axis is "communication overhead (%)" under a "processing" caption with an undefined baseline, and a model with enough free parameters would match any smooth curves and show nothing, so we did not fit one. To explore what such effects could do we built our own OMNeT++ model of the paper's Table II setting (research/reference_replication/). Its inputs include several guesses (adversary rate, server enrolment, arrival window), so we treat it as a sensitivity analysis and not as evidence about the authors' simulation or figures. As an illustration of that dependence only: over the guessed inputs we varied, the mean authentication latency of the proposed scheme at 1 000 devices ranges from about 2 ms to 5.6 s, so a rising curve like Fig. 1 is possible but its size is not determined by the paper's parameters. Consequently, every claim below about processing cost rests on the paper's Table V and our measured constants, not on Fig. 1, and we make no claim about Fig. 1's correctness or about the 21% figure beyond its non-derivability from the tables.

C. Measured primitive costs

Table I compares the paper's constants with ours; Fig. 1 plots the ESP8266 values. On the ESP8266 at

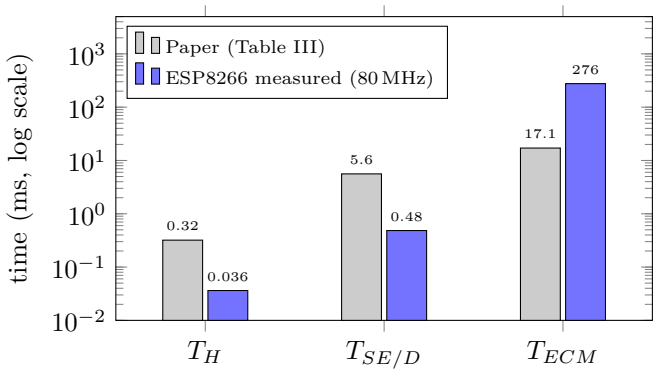


Fig. 1. Assumed versus measured per-operation time on the ESP8266 ($T_{SE/D}$ is aes_big GCM; aes_ct gives 1.15 ms).

80 MHz a P-256 scalar multiplication takes 276 ms (the fastest implementation available in its BearSSL core; generic code takes 624 ms and X25519 131 ms), sixteen times the assumed 17.1 ms, while SHA-1 and AES-GCM are 9× and 5–12× faster than assumed. The ratio that decides most rankings, T_{ECM}/T_H, is 7 600 on the device against the paper's 53. Timing spread is negligible on the microcontroller (the 25th and 75th percentiles equal the median), and ratios are identical at 160 MHz.

D. Recomputed ranking

Against ECC-based schemes the proposed scheme is far cheaper than the paper states. On the ESP8266 at 80 MHz (device plus server, AES-GCM, aes_big/aes_ct) it costs 2.1/4.7 ms, whereas Sadhukhan, Shuai and Cheng–Le cost 279–553 ms, Challa 2 487 ms and Poorambage 5 251 ms, i.e. about 60× (Sadhukhan against aes_ct) to 2 500× (Poorambage against aes_big) more. (Challa and Wazid include the fuzzy-extraction time T_{fe}, which we did not measure and keep at the paper's value.)

Against the hash-only rows the outcome depends on whether the session KDF is counted. Table V has three such rows: Turkanović et al., Dhillon–Kalra, and the row labelled "Jiang" whose source we could not identify (Section IV-A). The first two are confirmed hash-and-XOR-only schemes by the independent tabulations of [4], [9]. The proposed scheme beats a hash-only row only if $r = T_{SE/D}/T_H$ is below roughly 2.0–4.5, depending on the cost column; the paper's own constants give $r = 17.5$. With Table V as printed (V1), the hash-only rows are cheaper in all 12 AES-GCM sensitivity cells on the ESP8266 at both clocks ($r = 6.7$–31.6, i.e. 3.8–10.8× cheaper), in 11 of 12 on the Graviton2 core, and in 6 of 12 on the laptop. But measured HKDF-SHA256 costs 6–17 T_H on every platform, against 5–7 T_H for the hash-only device columns as printed (5–6 once Turkanović et al.'s swapped device and server counts are corrected), so a device that must run the session KDF already costs more than they do before any encryption. Table II shows that counting the KDF (V2–V4) makes all three rows cheaper than the proposed scheme in every column on every platform we measured, including OpenSSL's hardware AES and SHA (GCM 78 ns, SHA-1 38 ns, HKDF 232 ns per call); on the ESP8266 the gap is 3.8–13.7× across V1–V4. The ranking flips only if $T_{SE/D}$ is read as unauthenticated AES (one block or CTR without a MAC), which drops the MAC the scheme requires. The operation-count model also under-predicts the implementation: it gives 2.4 ms of device compute (aes_ct) where we measure 5.3 ms.

Jiang et al. as published is not a hash-only scheme. Its authentication phase alone needs $7T_H + 4T_C$ (≈ 131 ms with its own $T_C = 32.40$ ms), and login plus authentication need $11T_H + 6T_C$; on the ESP8266 the latter costs about 195 ms against 1–35 ms for the proposed scheme. It would be cheaper than the proposed scheme only if T_C were below a crossover of 0.14–5.2 ms (0.28–0.49 ms on the 80 MHz ESP8266, depending on the reading of the proposed scheme), which we did not measure. The conclusion that the proposed scheme costs more than its hash-only competitors therefore stands firmly for Turkanović et al. and Dhillon–Kalra, and for the unidentified "Jiang" row only to the extent that Table V's counts are right for it.

E. Protocol-level observations

No forward secrecy. The session key depends on $k^\star$, N_i, N_j and t (Eq. 16); N_i, N_j and t travel encrypted only under $k^\star$. An attacker who later obtains $k^\star$ recovered all 5 of 5 past session keys from recorded M_1/M_2 alone. Chang and Le's Table IV [9] lists no forward secrecy for Turkanović et al. either, so against that scheme the comparison is like for like; their second protocol and Jiang et al. claim it, so against the ECC-based schemes it is not.

ECC is not used online, and is nominal in the Section III model. Eq. 8 defines λ_X as a hash reduced modulo n, not a curve operation; the public point $P_X = \lambda_X G$ is set only optionally and Eqs. 9–22 never use it. (Section II's registration does run an ECDH exchange with the trusted authority, Eqs. 1–5; it is offline, belongs to the other description of the protocol, and we did not implement it.) Every online operation is a hash, KDF, encryption or MAC.

Server cost grows with the devices to be resolved. Because PID $= H_2(\lambda \,\|\, t)$ is unlinkable, the server cannot index by pseudonym and must probe (devices × epoch window). On the laptop one resolution took 4.8 ms at 1 000 and 23.9 ms at 5 000 devices (a 13-epoch window), against the constant server cost of Table V. A precomputed table avoids the compute but needs $1\,000 \times 13 \times 32$ B ≈ 416 KB at 1 000 devices against the paper's 320-bit server storage (Table VI): either way the scheme pays $O(D \cdot W)$ for D devices to resolve and window W. The paper does not say how many devices a server must resolve, so this is a scaling observation, not a magnitude.

V. Discussion and Threats to Validity

What the evidence supports. The paper's advantage over ECC-based schemes is real and larger than reported,

TABLE I
Paper constants versus measured medians. GCM: 56-byte AES-256-GCM including key setup, aes_big/aes_ct. P-256: fastest BearSSL implementation.

	Paper	ESP8266 80 MHz	ESP8266 160 MHz	Graviton2 (t4g.nano)
T_H (SHA-1)	0.32 ms	36 μs	18 μs	0.23 μs
$T_{SE/D}$ (GCM)	5.6 ms	0.48/1.15 ms	0.24/0.57 ms	2.0/6.9 μs
T_{ECM} (P-256)	17.1 ms	276 ms	138 ms	1.75 ms
HKDF-SHA256	—	629 μs	315 μs	3.2 μs

TABLE II
Number of the three hash-only rows of Table V that cost less than the proposed scheme (device+server total; GCM with cached key, aes_big; V1–V4 as defined in Section III).

Platform	V1	V2	V3	V4
ESP8266, 80 and 160 MHz (BearSSL)	3/3	3/3	3/3	3/3
Graviton2 t4g.nano (BearSSL)	3/3	3/3	3/3	3/3
Laptop, BearSSL software	3/3	3/3	3/3	3/3
Laptop, OpenSSL hardware AES/SHA	1/3	3/3	3/3	3/3

but those schemes may provide different properties (forward secrecy, Section IV-E). Its processing advantage over hash-only schemes is not supported once the session KDF that its own equations require is counted. Its communication and storage claims cannot be checked, because the paper's tables and text disagree and the competitor rows cannot all be traced to their sources. Its simulated figures are neither confirmed nor refuted here.

Threats to validity. We measured one microcontroller class (32-bit); the paper's targets include 8- and 16-bit parts, and we did not measure an MCU with AES or SHA hardware or energy. The ESP8266 timings were identical to the microsecond on two boards and across repeated runs; the laptop's absolute times shifted by 1.3–2.3× in Low Power Mode, so we rely on ratios there and recorded the power state of the rerun. Primitive costs are BearSSL's and OpenSSL's; other libraries shift constants, although the KDF result depends on HKDF costing several hash operations, not on absolute speed. A hardware HKDF/HMAC engine could change it. Competitor costs use the operation counts of [1]. We checked them against Chang and Le, Dhillon and Kalra, and Jiang et al.; Turkanović et al. is confirmed only through the two papers that tabulate it, and we had no source for the "Jiang" and "Shuai" rows or for any bit-size or storage entry. T_C and T_{fe} were not measured, and our own OMNeT++ model is a sensitivity analysis only (Section IV-B). The elliptic-curve benchmark ran on a temporary heap stack on the ESP8266 (the 4 KB task stack is too small), with outputs verified by checksum. The t4g.nano is a shared-host cloud VM pinned to one core, so its absolute numbers are indicative only, and it has no hardware-crypto measurement. Tables were transcribed from rendered PDFs and figure values read by eye. Our implementation is one reading of a specification that leaves the epoch and replay window undefined (it is held in RAM in the Python reference; the firmware persists it to flash). We have not contacted the authors.

VI. Conclusion and Future Work

Measured on real hardware, the protocol's cost model is dominated by two things the paper's constants get wrong: an ECC scalar multiplication on a small MCU is sixteen times slower than assumed, and the session-key derivation is several times more expensive than a hash. The first makes the scheme look better against ECC-based schemes than the paper claims; the second makes it look worse against hash-only schemes. The paper's stated percentages cannot be derived from its own data, and its processing figure cannot be reproduced from published information. Future work: obtain the authors' simulation scenario and rerun Fig. 1 against it; measure an MCU with hardware AES and SHA and a Raspberry Pi 3; trace the unidentified competitor rows and verify the remaining entries against the original papers; implement and time Turkanović et al. end to end on the microcontroller rather than modelling it; and complete the TA-assisted key-revocation extension we proposed earlier.

Reproducibility

The Python reference implementation, the C device core, the ESP8266 firmware and benchmark, the audit scripts and all raw data (including output checksums) are in https://github.com/jaykerkar0405/hybrid-ecc-iot-auth under firmware/esp8266/, research/paper_audit/ and research/reference_replication/; the file research/paper_audit/FINDINGS.md lists the commands.

References

[1] A. Al-Rasheed, R. Khan, A. Ahmad, A. Saeed, F. Alturise, and S. Alkhalaf, "A low-cost hybrid elliptic curve cryptography authentication protocol for trustworthy Internet of Things communication," IEEE Trans. Consum. Electron., vol. 72, no. 2, pp. 4483–4491, May 2026.

[2] P. Porambage, C. Schmitt, P. Kumar, A. Gurtov, and M. Ylianttila, "Two-phase authentication protocol for wireless sensor networks in distributed IoT applications," in Proc. IEEE Wireless Commun. Netw. Conf. (WCNC), Apr. 2014, pp. 2728–2733.

[3] M. Turkanović, B. Brumen, and M. Hölbl, "A novel user authentication and key agreement scheme for heterogeneous ad hoc wireless sensor networks, based on the Internet of Things notion," Ad Hoc Netw., vol. 20, pp. 96–112, Sep. 2014.
[4] P. K. Dhillon and S. Kalra, "A lightweight biometrics based remote user authentication scheme for IoT services," J. Inf. Secur. Appl., vol. 34, pp. 255–270, Jun. 2017.
[5] Q. Jiang, F. Wei, S. Fu, J. Ma, G. Li, and A. Alelaiwi, "Robust extended chaotic maps-based three-factor authentication scheme preserving biometric template privacy," Nonlinear Dyn., vol. 83, no. 4, pp. 2085–2101, Mar. 2016.
[6] S. Challa et al., "Secure signature-based authenticated key establishment scheme for future IoT applications," IEEE Access, vol. 5, pp. 3028–3043, 2017.
[7] D. Sadhukhan, S. Ray, G. P. Biswas, M. K. Khan, and M. Dasgupta, "A lightweight remote user authentication scheme for IoT communication using elliptic curve cryptography," J. Supercomput., vol. 77, no. 2, pp. 1114–1151, Feb. 2021.
[8] M. Fakroon, M. Alshahrani, F. Gebali, and I. Traore, "Secure remote anonymous user authentication scheme for smart home environment," Internet of Things, vol. 9, Art. no. 100158, Mar. 2020.
[9] C.-C. Chang and H.-D. Le, "A provably secure, efficient, and flexible authentication scheme for ad hoc wireless sensor networks," IEEE Trans. Wireless Commun., vol. 15, no. 1, pp. 357–366, Jan. 2016.
[10] T. Pornin, "BearSSL, a smaller SSL/TLS library," https://bearssl.org. The ESP8266 Arduino core ships a port of BearSSL.
[11] OpenSSL Project, "OpenSSL cryptography and SSL/TLS toolkit," https://www.openssl.org.
[12] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, May 2010.
[13] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D, Nov. 2007.